

## Operating Systems - Lecture 1: Introduction to Operating Systems

What is an Operating System (Sistemi Operativ)?

An Operating System (OS) is system software that manages hardware and software resources. It acts as an intermediary between users and the hardware.

👉 Think of it like a manager in a company—without one, employees (hardware) wouldn't know what tasks to do, and clients (users) wouldn't get their work done.

Core Responsibilities of an OS

- ✓ Manages Processes - Decides which programs run and for how long.
- ✓ Controls Memory - Allocates and tracks RAM usage.
- ✓ Handles Input/Output (I/O) - Manages files, devices, and communication.
- ✓ Enforces Security - Restricts unauthorized access.
- ✓ Provides a User Interface - GUI (Graphical) or CLI (Command Line).

💡 Example:

When you open Google Chrome:

The OS allocates memory to Chrome.

It schedules CPU time so the browser runs smoothly.

It handles I/O requests (mouse clicks, typing).

It secures network access to prevent unauthorized connections.

Why Do We Need an OS?

A computer without an OS is just a pile of circuits. The OS coordinates everything, making it possible to run multiple programs at once without conflict.

Imagine you're running Spotify + Word + YouTube:

The OS ensures that Spotify doesn't freeze Word.

It shares CPU time between tasks.

It prioritizes processes, so an urgent task (like a keyboard press) happens instantly.

Without an OS, every program would have to manually manage hardware, which is impractical.

The 4 Main Components of a Computer System

A computer is built from four essential parts:

- 🖥️ 1. Hardware (Hardueri) - The physical components like CPU, RAM, and disk.
- ⚙️ 2. Operating System (Sistemi Operativ) - Manages and controls everything.
- 📁 3. Application Programs (Programet Aplikative) - The software users interact with (e.g., browsers, games).
- 👤💻 4. Users (Shfrytëzuesit) - The people using the system.

💡 Example:

A Linux server running a website:

The OS schedules HTTP requests.

Memory management prevents crashes.

File system operations ensure smooth database storage.

The Kernel - The Brain of the OS

The Kernel is the core of the OS. It is always running and controls the most critical operations.

Two main types of kernels:

◆ Monolithic Kernel (Linux, Windows) - Everything runs together, fast but harder to maintain.

◆ Microkernel (QNX, Minix) - Small core functions in the kernel, other services run separately, more stable but slower.

💡 Example:

A Windows system crash (Blue Screen of Death) is often a kernel failure, meaning the core system became unstable.

The Boot Process - How a Computer Starts Up

When you press the power button:

1️⃣ BIOS/UEFI loads firmware (stored in ROM).

2️⃣ Bootstrap Program initializes the OS (Bootloader like GRUB or Windows Boot Manager).

3️⃣ Kernel is loaded into RAM, and system services start running.

4️⃣ The system is ready for user interaction.

💡 Example:

If a system doesn't boot, it might be due to corrupt bootloader files, requiring recovery.

Storage Hierarchy - Managing Data Efficiently

Storage is divided into layers based on speed and volatility:

🔗 Registers (CPU L1/L2 Cache) - Fastest storage, holds CPU instructions for immediate execution.

📄 RAM (Memory) - Fast, but volatile, meaning it loses data when powered off. Used for temporary program execution.

🌀 SSD (NVMe, SATA) - Fast, non-volatile storage where the OS and software are installed.

💾 HDD (Magnetic Disks) - Slower than SSDs, used for large-scale file storage.

☁ Cloud/External Storage - Slowest but permanent, used for backups and remote access.

💡 Example:

When you copy a file from an SSD to a USB, you notice a speed difference—this happens due to different data transfer rates in storage types.

## Caching - Speeding Up Performance

Caching stores frequently accessed data in faster memory to reduce wait times.

### Types of Caching:

- ◆ CPU Cache (L1/L2) - Speeds up execution of instructions.
- ◆ Disk Cache - Preloads file system data for quick retrieval.
- ◆ Browser Cache - Saves websites for faster loading.

### Example:

When you visit a website, the browser stores static content (images, styles) locally so it loads faster next time.

## Device Drivers - How OS Communicates with Hardware

The OS doesn't directly control hardware. Instead, it uses Device Drivers, which act as translators between the OS and peripherals.

### Types of Drivers:

- ◆ Character Drivers - Handle keyboards, mice, serial ports.
- ◆ Block Drivers - Manage storage devices (HDD, SSD).
- ◆ Network Drivers - Control WiFi, Ethernet, Bluetooth.

### Example:

When you plug in a new printer, the OS installs a driver to recognize and use it properly.

## Process Management - Handling Multiple Programs at Once

A process is a program in execution, and the OS manages these efficiently.

### Process Life Cycle:

- 1 New - Process is created.
- 2 Running - Actively using CPU.
- 3 Waiting - Suspended for I/O or an event.
- 4 Ready - Waiting for CPU allocation.
- 5 Terminated - Process completes execution.

### Example:

When you watch a video while downloading a file, the OS ensures both tasks run smoothly by switching CPU time between them.

## Virtualization - Running Multiple OS on One System

Virtualization allows multiple OS instances to run on the same physical machine.

### Types of Virtualization:

- ◆ Hypervisors (VMware, KVM, Xen) - Create full virtual machines (VMs).
- ◆ Containers (Docker, Kubernetes) - Isolated environments sharing the same OS kernel.

### Example:

Cloud platforms like AWS & Google Cloud run thousands of VMs using hypervisors, allowing multiple users to share resources without interference.

Why This Matters?

Understanding the structure of an OS helps in:

- ✓ Optimizing system performance (managing resources efficiently).
- ✓ Troubleshooting issues (boot errors, system crashes).
- ✓ Developing software that interacts well with system resources.

-----  
-----  
-----

## Operating Systems - Lecture 2: Structure of Operating Systems

### Services Provided by the Operating System

An Operating System (OS) is responsible for providing a range of services that make computer usage efficient, secure, and user-friendly. These services manage hardware, allow software execution, and ensure smooth system operation.

 User Interface (UI) - Almost all OS provide a user interface, either Graphical (GUI) like Windows/macOS or Command-Line (CLI) like Linux Terminal.

 Program Execution - The OS must be able to load programs into memory, execute them, and terminate them either normally or abnormally (due to an error like segmentation fault).

 I/O Operations - Programs often require access to files or I/O devices (keyboards, printers, USBs). The OS handles these operations so programs don't need direct hardware control.

 File System Management - The OS must allow applications to read, write, delete, and organize files and directories. It also manages file permissions and access control.

 Process Communication - Some applications need to exchange data with each other. The OS provides Inter-Process Communication (IPC) mechanisms, such as Shared Memory and Message Passing.

 Error Detection and Handling - The OS monitors the system for CPU, memory, and I/O device failures. If a problem is detected, it must take corrective actions to maintain stability.

 Resource Allocation - In multi-user or multi-tasking systems, the OS distributes CPU time, memory, and I/O bandwidth efficiently among running programs.

📊 Logging (Accounting) - The OS keeps track of which users or processes are using system resources, ensuring auditability and performance monitoring.

## 🔒 Protection & Security

Protection: Ensures that unauthorized programs cannot access restricted files or memory areas.

Security: Implements user authentication (passwords, biometrics) and prevents unauthorized access to I/O devices.

💡 Example: When multiple users access a Linux server, the OS enforces file permissions (read/write/execute) so that one user cannot modify another user's files.

## System Calls - How Applications Interact with the OS

System Calls provide an interface between user applications and the OS kernel. When a program needs to perform file operations, memory allocation, or process control, it makes a system call instead of directly accessing hardware.

## 💠 System Call Workflow:

A program requests a service from the OS by calling a system function. The system call is mapped to a unique number and passed to the OS kernel. The OS executes the requested operation (e.g., opening a file). The result is returned to the application.

💡 Example:

When a Python script opens a file using `open("data.txt", "r")`, the OS executes a system call like `sys_open()`, handling file access at a lower level.

## Three Common OS APIs for System Calls

Operating systems provide Application Programming Interfaces (APIs) that allow developers to use system calls without dealing directly with the kernel. The three most widely used APIs are:

✚ Win32 API - Used for system calls in Windows OS.

✚ POSIX API - Standard for UNIX-based systems (Linux, macOS).

✚ Java API - Used in JVM-based applications, providing cross-platform functionality.

💡 Example: If a C program on Linux needs to create a process, it calls `fork()`, which is defined in the POSIX API, and the OS executes it at the kernel level.

## Passing Parameters to the OS in System Calls

When a system call is made, parameters must be passed to the OS. There are three primary ways to do this:

① Registers - Parameters are placed in CPU registers (fastest).

② Memory Blocks - Parameters are stored in a memory block, and only the memory address is passed to the OS.

📁 Stacking - Parameters are pushed onto the process stack and retrieved when needed (used in function calls).

💡 Example:

When calling `write(fd, buffer, size)`, the OS may place `fd` in a register, store `buffer` in memory, and push `size` onto the stack.

### Types of System Calls

System calls can be grouped into five categories:

📁 File Management - Creating, opening, reading, writing, and deleting files.

🖨 Device Management - Communicating with devices (printers, hard drives).

📊 Information Maintenance - Getting or setting system information (e.g., `getpid()`).

🔄 Process Communication - Sending and receiving messages between processes.

🔒 Protection & Security - Managing user access and file permissions.

💡 Example:

The `exec()` system call replaces a running process with a new program, often used in command-line shells.

### System Programs - Essential OS Tools

System programs provide a user-friendly environment for development and system management. These are not part of the OS kernel but interact with it.

📁 File Manipulation - Copying, renaming, deleting files (`mv`, `rm` in Linux).

📊 Status Information - Displaying system statistics (`top`, task manager).

💻 Programming Support - Compilers, debuggers (GCC, Visual Studio).

🔗 Program Execution - Launching applications (`bash`, `cmd.exe`).

🌐 Inter-Process Communication - Managing network connections.

🕒 Background Services - Daemons and background processes (`cron`, Windows Services).

💡 Example:

In Linux, `grep` is a system program used for searching text within files, but it relies on system calls like `open()` and `read()` to access data.

### How Programs Are Compiled and Linked

When you write source code, it must go through multiple steps before execution.

📌 Relocatable Object File - After compilation, source code is converted into object files that can be loaded into memory at different locations.

🔑 Linker - Combines multiple object files into a single binary executable and links necessary libraries.

🔑 Loader - The OS loads the binary executable into memory and prepares it for execution.

💡 Example:

In C programming, when you compile gcc main.c, the linker combines standard libraries (libc) with your code before execution.

### Memory Relocation & ABI

When a program is executed, the OS must assign final memory addresses for code and data. This process is called relocation.

🔑 Application Binary Interface (ABI) - Defines how compiled programs interact with the OS and hardware, ensuring compatibility across different architectures (e.g., x86 vs. ARM).

💡 Example:

An executable compiled for Windows (x86-64 ABI) will not run on macOS (ARM64 ABI) without recompilation.

### Design & Implementation Goals of an OS

A well-designed OS must balance usability, performance, and security.

👤 User Goals - The OS should be easy to use, secure, and fast.

⚙️ System Goals - The OS should be flexible, reliable, and efficient while minimizing errors.

💡 Example:

Windows prioritizes user experience, while Linux is optimized for stability and efficiency.

### Policy vs. Mechanism in OS Design

🔑 Policy - Defines what should be done (e.g., prioritize real-time processes).

🔑 Mechanism - Defines how it should be done (e.g., implement Round-Robin scheduling).

💡 Example:

In CPU scheduling, policy decides which process gets priority, while mechanism implements the actual algorithm.

### UNIX System Structure

UNIX consists of two main components:

◆ System Programs - User-level utilities and command-line tools.

◆ Kernel - Provides system calls, file system management, CPU scheduling, and memory management.

### Layered OS Architecture

A layered OS is built in multiple levels, where:

The lowest layer (Layer 0) interacts with hardware.

The highest layer provides the User Interface (UI).

Microkernel Architecture

In Microkernel systems, only essential functions (process communication, memory management) run in kernel mode, while other OS services run in user mode, improving stability and modularity.

💡 Example: macOS and Windows NT use a hybrid microkernel for better performance.

-----  
-----  
-----

Operating Systems - Lecture 3: Processes

What is a Process?

A process is an instance of a program in execution. It consists of:

Code (Text Section) - The actual instructions of the program.

Program Counter - Keeps track of the next instruction to execute.

Stack - Stores function calls, local variables.

Heap - Stores dynamically allocated memory during runtime.

Data Section - Contains global variables.

💡 Example:

When you open Google Chrome, each tab is treated as a separate process so that if one tab crashes, it doesn't affect the others.

Process Control Block (PCB) - The Identity of a Process

Each process has a Process Control Block (PCB), which stores critical information needed by the OS to manage it. The PCB contains:

Process State - Whether the process is new, running, waiting, ready, or terminated.

Program Counter - Tracks the next instruction to be executed.

CPU Registers - Stores values of CPU registers for the process.

CPU Scheduling Information - Includes priority, queue pointers.

Memory Management Info - Memory allocated to the process.

Accounting Information - Tracks CPU usage, execution time.

I/O Status Information - Lists open files, assigned I/O devices.

💡 Example:

If a process is suspended, its PCB stores all execution details so it can be resumed later without loss.

Context Switching - How the OS Switches Between Processes

A context switch occurs when the CPU stops executing one process and switches to another.

The OS saves the state (registers, program counter) of the current process in its PCB.

The OS loads the PCB of the next process to be executed.

Execution resumes from where it was paused.



### 💡 Example:

If you're typing in Word while listening to music, the OS rapidly switches between Word and Spotify so both applications seem to run simultaneously.

How Process Execution Works in Mobile OS (iOS vs. Android)

📱 iOS - Only one foreground process runs at a time, controlled by the UI. Background processes exist but are heavily restricted to save battery.

🤖 Android - Runs both foreground and background processes with fewer restrictions. Background processes can use services to continue running even if they're not visible.

### 💡 Example:

On iOS, Spotify stops playing if it doesn't have background audio permissions. On Android, a music player can continue running without needing UI activity.

Process Creation & Termination

🔗 Creating a Process: A parent process spawns child processes, forming a process tree. Each process is uniquely identified by a Process Identifier (PID).

### ✗ Terminating a Process:

A process finishes execution and calls `exit()`.

A parent process can terminate a child process using `abort()`.

If a parent is terminated, the OS might also terminate its child processes.

### 💡 Example:

In Linux, when you open VS Code, it creates multiple child processes for rendering, extensions, and debugging. If you close VS Code, all child processes terminate automatically.

Google Chrome's Multi-Process Architecture

Chrome runs in multiple separate processes to enhance security and stability. The three main process types are:

Browser Process - Handles the main UI, network requests, tabs management.

Renderer Process - Runs web pages, executes JavaScript in a sandboxed environment.

Plug-in Process - Handles third-party plugins like Flash (deprecated).

### 💡 Example:

If one tab crashes, only the renderer process for that tab is affected, not the entire browser.

Independent vs. Cooperating Processes

💎 Independent Processes - Do not affect or interact with other processes.

💎 Cooperating Processes - Share data and depend on each other's execution.

### 💡 Example:

Independent: Playing a video while downloading a file.

Cooperating: A compiler breaking down tasks like parsing, optimization, and code generation, where each stage depends on the previous one.

Interprocess Communication (IPC) - How Processes Communicate

Cooperating processes need to exchange data. The OS provides two main IPC models:

Shared Memory - Processes share a common memory space to exchange information (fast, but requires synchronization).

Message Passing - Processes send and receive messages via the OS (slower, but more secure).

💡 Example:

A web server (Apache) uses shared memory for fast caching, while email services use message passing for communication.

Producer-Consumer Problem - A Common IPC Scenario

A Producer Process creates data, while a Consumer Process uses it.

◆ Unbounded Buffer - The producer never waits since there is no limit on storage.

◆ Bounded Buffer - The producer must wait if the buffer is full, and the consumer must wait if the buffer is empty.

💡 Example:

Unbounded: A log file where logs are continuously written.

Bounded: A print queue where jobs are processed one at a time.

Message Passing in IPC

Instead of sharing memory, processes can communicate by sending and receiving messages.

◆ Direct Communication - Processes must specify each other explicitly:

send(P, message) - Send a message to process P.

receive(Q, message) - Receive a message from process Q.

◆ Indirect Communication - Messages are placed in a mailbox (queue) with a unique ID. Processes retrieve messages from this mailbox instead of direct communication.

💡 Example:

A chat application uses direct communication, while an email server stores messages in a mailbox until the recipient checks them.

Blocking vs. Non-Blocking Communication

Blocking (Synchronous) - The sender waits until the message is received.

Non-blocking (Asynchronous) - The sender sends the message and continues execution.

◆ Blocking (Synchronous):

Blocking Send - The sender is paused until the message is received.

Blocking Receive - The receiver is paused until a message arrives.

### ◆ Non-Blocking (Asynchronous):

Non-Blocking Send - The sender sends the message and doesn't wait.

Non-Blocking Receive - The receiver either gets a valid message or an empty response.

💡 Example:

Blocking: A phone call - The caller must wait until the receiver picks up.  
Non-Blocking: Sending a text - The sender continues with other tasks after sending.

Buffering - How Messages Are Stored Before Delivery

The OS maintains queues (buffers) to temporarily store messages. There are three buffering types:

◆ Zero Capacity - No buffering. The sender must wait until the receiver is ready.

◆ Bounded Capacity - The queue has a limited size; if full, the sender must wait.

◆ Unbounded Capacity - The queue has no size limit, so the sender never waits.

💡 Example:

A printer queue is a bounded buffer since only a limited number of print jobs can be stored at once.

Pipes - Simple Data Transfer Between Processes

A pipe is a special mechanism that allows processes to send data in one direction like a pipeline.

◆ Ordinary Pipes - Exist only between a parent and child process.

◆ Named Pipes - Allow unrelated processes to communicate.

💡 Example:

In Linux, the command `ls | grep "file"` creates an ordinary pipe where the output of `ls` is passed as input to `grep`.

Sockets - Network Communication Between Processes

A socket is an endpoint for communication over a network. It allows processes on different computers to send and receive data.

💡 Example:

When you browse the web, your browser opens a socket connection to a web server, sending an HTTP request and receiving a response.

-----  
-----  
-----

Operating Systems - Lecture 4: Threads, Concurrency, and CPU Scheduling  
Concurrency vs. Parallelism

Concurrency and parallelism are two key concepts in modern computing, but they do not mean the same thing.

◆ Concurrency - Supports multiple tasks that make progress simultaneously but may not be executing at the exact same time.

◆ Parallelism - The system executes multiple tasks at the same time on different CPU cores.

💡 Example:

Concurrency: A single-core processor switches between multiple tasks quickly, making them appear to run at the same time.

Parallelism: A multi-core processor actually executes multiple tasks at the same time.

Types of Parallelism

◆ Data Parallelism - Splits large data into smaller chunks, distributing them across multiple cores, each executing the same operation.

◆ Task Parallelism - Splits the computation into multiple tasks, each handled by a separate thread.

💡 Example:

Data Parallelism: If you're processing an image, each core handles a portion of the pixels.

Task Parallelism: In a web server, one thread handles requests, another handles database queries, and another manages file uploads.

User vs. Kernel Threads

Threads can be managed at two levels:

◆ User Threads - Managed by user-space libraries, the OS kernel is unaware of them.

Examples: POSIX Pthreads, Windows Threads, Java Threads

Faster but may not take full advantage of multi-core CPUs.

◆ Kernel Threads - Managed directly by the OS kernel, allowing full utilization of system resources.

Slower due to OS overhead, but more efficient for multi-core processing.

💡 Example:

A Java Virtual Machine (JVM) uses user threads, while Linux Kernel Threads control device drivers and background services.

Multithreading Models

To efficiently manage user threads and kernel threads, the OS can implement one of three models:

❏ Many-to-One - Multiple user threads are mapped to a single kernel thread.

Limitation: If one thread makes a blocking system call, the entire process halts.

❏ One-to-One - Each user thread is mapped to a unique kernel thread.

Advantage: True parallel execution, but consumes more OS resources.

③ Many-to-Many - Multiple user threads are mapped to multiple kernel threads.

Best balance between performance and resource efficiency.

💡 Example:

Many-to-One: Early Java Threads (single-threaded systems).

One-to-One: Windows Threads, Linux pthreads.

Many-to-Many: Modern Linux threading models, hybrid thread pools.

CPU Scheduling - How the OS Decides Which Process Runs

A CPU scheduler selects processes from the ready queue and assigns them to the CPU. This decision happens when:

① A process moves from running to waiting (e.g., waiting for I/O).

② A process moves from running to ready (preempted by another process).

③ A process moves from waiting to ready (I/O is completed).

④ A process terminates (finishes execution).

◆ Non-Preemptive Scheduling - Once a process gets the CPU, it keeps it until it voluntarily releases it (case 1 and 4).

◆ Preemptive Scheduling - The OS forces a process to give up the CPU if a higher-priority process arrives.

💡 Example:

Non-Preemptive: A cashier helps one customer fully before moving to the next.

Preemptive: A hospital emergency room treats critical patients first, regardless of arrival time.

Dispatcher - The Component That Switches Processes

The dispatcher is responsible for giving control of the CPU to the selected process. It must:

✓ Save the state of the old process.

✓ Load the state of the new process.

✓ Switch execution to the new process.

◆ Dispatch Latency - The time taken by the dispatcher to switch from one process to another.

💡 Example:

A fast dispatcher means smoother multitasking and lower CPU idle time.

Key CPU Scheduling Criteria

A good CPU scheduling algorithm optimizes multiple factors:

◆ CPU Utilization - Ensures the CPU is busy as much as possible.

◆ Throughput - Maximizes the number of processes completed per unit time.

◆ Turnaround Time - The total time from process creation to completion.

◆ Waiting Time - Time spent in the ready queue before execution.

◆ Response Time - The time between request submission and first response.

💡 Example:

A real-time system prioritizes low response time.

A batch processing system prioritizes high throughput.

CPU Scheduling Algorithms

✂ First-Come, First-Served (FCFS)

Processes are executed in order of arrival.

Non-preemptive - Once a process starts, it runs to completion.

Problem: If a long process arrives first, shorter processes wait longer (convoy effect).

✂ Shortest Job First (SJF)

The CPU is assigned to the process with the shortest burst time.

Can be preemptive (Shortest Remaining Time First) or non-preemptive.

Problem: Starvation - Longer processes may never execute if shorter ones keep arriving.

✂ Round Robin (RR)

Each process gets a fixed time slice (quantum), usually 10-100ms.

If a process does not finish within its time slice, it moves to the back of the queue.

Fair for interactive systems, but switching overhead increases with smaller time slices.

✂ Priority Scheduling

Each process has a priority value; CPU is allocated to the highest-priority process.

Can be preemptive or non-preemptive.

Problem: Starvation - Low-priority processes may never get CPU time.

Solution: Aging - Gradually increase priority of long-waiting processes.

✂ Multilevel Queue Scheduling

The ready queue is divided into multiple queues, each with a different priority level.

Example: System processes, Interactive processes, Batch jobs.

Once a process is assigned to a queue, it stays in that queue.

✂ Multilevel Feedback Queue (MLFQ)

Similar to Multilevel Queue, but processes can move between queues based on execution behavior.

New processes start at higher priority but are moved down if they use too much CPU.

Long-waiting processes are moved up to avoid starvation.

💡 Example:

A background task (file indexing) starts with low priority but moves higher if it hasn't been executed for a long time.

Why CPU Scheduling Matters?

Efficient CPU scheduling improves system performance, reduces waiting times, and ensures fair CPU distribution. The choice of algorithm depends on the use case:

- ✓ FCFS - Best for simple batch jobs (low overhead, but inefficient).
- ✓ SJF - Best for minimizing turnaround time (but risks starvation).
- ✓ Round Robin - Best for time-sharing systems (interactive workloads).
- ✓ Priority Scheduling - Best for real-time and critical applications (but requires aging).
- ✓ MLFQ - Best for complex workloads (dynamic adjustment based on behavior).

-----  
-----  
-----

## Operating Systems - Lecture 5: Process Synchronization

### Why Process Synchronization is Important?

In a multitasking environment, multiple processes access shared resources. Without proper synchronization, race conditions and data inconsistencies can occur.

### 💡 Example:

Banking System: Two processes updating the same bank account simultaneously may cause an incorrect balance.

File System: Two processes writing to the same file without synchronization can lead to corrupt data.

To solve this, operating systems enforce synchronization using critical sections, locks, semaphores, and monitors.

### The Critical Section Problem

Each process has a critical section - a part of its code that modifies shared data. The OS must ensure that only one process enters this section at a time.

### ✂ Structure of a Process with a Critical Section:

- 1️⃣ Entry Section - The process requests access to the critical section.
- 2️⃣ Critical Section - The process executes code that modifies shared data.
- 3️⃣ Exit Section - The process exits the critical section, allowing others to enter.
- 4️⃣ Remainder Section - The process executes other non-critical operations.

### 💎 Three Requirements for a Solution:

- ✓ Mutual Exclusion - If a process is in the critical section, no other process can enter.
- ✓ Progress - If no process is in the critical section, others must be able to enter without indefinite delays.
- ✓ Bounded Waiting - A process cannot be blocked forever while waiting to enter the critical section.

#### 💡 Example:

If two people want to withdraw money from the same ATM, the system must ensure only one transaction happens at a time.

#### Hardware Synchronization Mechanisms

Operating systems use hardware-level synchronization to ensure atomic operations.

#### ◆ Locks & Interrupts

Single-Processor Systems: Use interrupt disabling to prevent context switches during critical operations.

Multi-Processor Systems: Use atomic hardware instructions to enforce synchronization.

#### ◆ Atomic Operations - Instructions that execute without interruption.

Test-and-Set (TAS) - Locks shared resources.

Compare-and-Swap (CAS) - Ensures changes occur only if no modifications were made by another process.

#### 💡 Example:

A spinlock uses atomic instructions to prevent multiple threads from modifying shared data simultaneously.

#### Mutex Locks - Ensuring Mutual Exclusion

A mutex (mutual exclusion) is a binary lock that restricts access to a shared resource. Only one process can hold the mutex at a time.

#### ◆ Operations on a Mutex:

🔒 acquire() - Locks the mutex. If another process holds it, the calling process waits.

🔓 release() - Unlocks the mutex, allowing another process to enter the critical section.

#### 💡 Example:

A printer spooler uses a mutex to ensure that only one print job is processed at a time.

#### Semaphores - Controlling Access to Shared Resources

A semaphore is a counter used to manage access to a resource by multiple processes. It controls how many processes can use a resource at the same time.

#### ◆ Two Types of Semaphores:

✓ Binary Semaphore (Mutex) - Only two values: 0 (locked) and 1 (unlocked).

✓ Counting Semaphore - Allows multiple processes to access a resource.

#### ◆ Operations on Semaphores:

🔒 wait() / P() - Decrements the semaphore count. If count is zero, the process waits.



🔒 `signal() / V()` - Increments the semaphore count, allowing waiting processes to proceed.

💡 Example:

Database Connection Pooling - A semaphore limits the number of active database connections.

Traffic Light System - Controls how many cars can pass at a given time.

Monitors - High-Level Synchronization Constructs

A monitor is a higher-level synchronization mechanism that groups shared variables, procedures, and synchronization primitives into a single module.

💎 How Monitors Work:

Only one thread can execute inside a monitor at a time.

Threads wait until a condition is met before proceeding.

The monitor handles automatic mutual exclusion, so no explicit locks are needed.

💡 Example:

A hotel reservation system using monitors ensures only one customer books a room at a time, preventing double bookings.

Differences Between Semaphores and Monitors

💎 `wait()` Behavior:

✓ Semaphores: If `wait()` is called and `count > 0`, no blocking occurs.

✓ Monitors: If `wait()` is called, the process blocks until the condition is met.

💎 `signal()` Behavior:

✓ Semaphores: Increments the semaphore count and unblocks a waiting process if necessary.

✓ Monitors: Only wakes up a waiting process, but does not modify the condition variable.

💡 Example:

A producer-consumer problem using monitors ensures that a producer waits if the buffer is full, and a consumer waits if the buffer is empty.

Blocking vs. Non-Blocking Synchronization

💎 Blocking (Synchronous) - A process waits until a condition is met before continuing execution.

💎 Non-Blocking (Asynchronous) - A process continues execution immediately, checking for the condition later.

💡 Example:

Blocking: A thread waits for a file to be downloaded before processing it.

Non-Blocking: A thread initiates the download and continues execution, checking later if the file is ready.

Buffering - How Messages Are Stored in Synchronization

To enable communication between processes, messages are stored in queues (buffers):

- ◆ Zero Capacity Buffer - The sender must wait until the receiver is ready.
- ◆ Bounded Capacity Buffer - The queue has a fixed limit; if full, the sender must wait.
- ◆ Unbounded Capacity Buffer - The queue can grow infinitely, so the sender never waits.

💡 Example:

A print queue is a bounded buffer - if too many print jobs are queued, new jobs must wait.

Pipes - Simple Data Communication Between Processes

A pipe is a unidirectional communication channel between two processes.

◆ Two Types of Pipes:

- ✓ Ordinary Pipes - Exist only between parent and child processes.
- ✓ Named Pipes (FIFOs) - Can be used between unrelated processes.

💡 Example:

In Linux, `ls | grep "file"` creates a pipe, passing the output of `ls` as input to `grep`.

Sockets - Synchronization Over a Network

A socket is an endpoint for network communication. It allows processes on different systems to send and receive messages.

💡 Example:

A web browser and a web server communicate through TCP sockets.

Why Process Synchronization Matters?

Process synchronization prevents race conditions, ensures data consistency, and avoids deadlocks. The choice of synchronization technique depends on the requirements of the system:

- ✓ Mutexes - Best for mutual exclusion of a single resource.
- ✓ Semaphores - Best for controlling multiple processes accessing shared resources.
- ✓ Monitors - Best for structured synchronization in object-oriented systems.
- ✓ Pipes & Sockets - Best for process communication and networking.

-----  
-----  
-----

Operating Systems - Lecture 6: Deadlocks

What is a Deadlock?

A deadlock occurs when a set of processes are stuck in a state where none of them can proceed because they are waiting for resources held by other processes. This situation leads to an infinite waiting condition.

💡 Example:

Imagine two cars on a narrow one-lane bridge coming from opposite directions. Neither car can move because each is waiting for the other to back up.

Four Necessary Conditions for Deadlock

A deadlock can only occur if all four of the following conditions hold simultaneously:

- ❑ Mutual Exclusion - Only one process at a time can use a resource.
- ❑ Hold and Wait - A process holds at least one resource while waiting for more resources.
- ❑ No Preemption - A resource can only be released voluntarily by the process that holds it.
- ❑ Circular Wait - A set of processes are waiting in a circular chain, where each process is waiting for a resource held by the next process in the chain.

💡 Example:

A system with three processes (P1, P2, P3) and three resources (R1, R2, R3):

P1 holds R1 and is waiting for R2 (held by P2).  
P2 holds R2 and is waiting for R3 (held by P3).  
P3 holds R3 and is waiting for R1 (held by P1).  
Since no process can proceed, a deadlock occurs.

Methods to Handle Deadlocks

To ensure a system never enters a deadlock state, the OS can use one of these three strategies:

- ❑ Deadlock Prevention - Modify the system to eliminate at least one of the four necessary conditions.
- ❑ Deadlock Avoidance - Dynamically detect and prevent unsafe resource allocations before they lead to deadlocks.
- ❑ Deadlock Recovery - If a deadlock occurs, detect it and forcefully resolve it.

Deadlock Prevention - Eliminating One of the Four Conditions

◆ Mutual Exclusion - If possible, design resources so that they can be shared instead of requiring exclusive access.

Example: A read-only file can be accessed by multiple processes at once.

◆ Hold and Wait - A process must request all resources at once, preventing it from holding one resource while waiting for another.

Example: Before a process starts execution, it must acquire all required resources in advance.

◆ No Preemption - Allow the system to forcibly take resources from a process if necessary.

Example: If a process is waiting for a resource that another process holds, the OS can forcefully take the resource away.

◆ Circular Wait - Impose a total ordering of resources so that processes must request them in a fixed sequence.

Example: If a process holds Resource A, it cannot request Resource B unless it releases Resource A first.

💡 Example:

A printer and scanner system follows a resource ordering rule:

If a process wants to use both, it must request the printer before the scanner.

This prevents circular wait conditions.

Deadlock Avoidance - Ensuring a Safe State

Unlike prevention, deadlock avoidance allows flexible resource allocation, but ensures the system never enters an unsafe state.

◆ Safe State Definition: A system is in a safe state if it can guarantee that all processes can finish execution.

◆ Banker's Algorithm (Deadlock Avoidance Technique)

Each process must declare its maximum resource needs upfront.

The OS simulates resource allocation before granting requests.

If granting the request leads to a safe state, it is approved. Otherwise, it is denied.

💡 Example:

A bank with \$100 in funds manages 3 loan requests:

Customer A: Needs \$30, already has \$10.

Customer B: Needs \$60, already has \$20.

Customer C: Needs \$40, already has \$10.

The OS grants loans only if it ensures all customers can eventually be served without getting stuck.

Deadlock Recovery - Handling Deadlocks When They Occur

If the OS does not prevent or avoid deadlocks, it must have a recovery mechanism in place.

◆ Three Approaches to Recovery:

❏ Process Termination (Killing a Process)

Select a victim process and terminate it to free resources.

Minimizes system disruption, but may cause data loss.

The OS must choose the victim carefully to minimize impact.

❏ Resource Preemption (Forcefully Taking Resources)

Temporarily remove resources from a process and reallocate them.

May require rolling back processes to previous states.  
Starvation risk: A process may be continuously preempted.  
📦 Process Rollback (Returning to a Safe State)

Instead of terminating, revert processes to earlier checkpoints.  
Requires saving process states periodically.  
Used in database systems and real-time applications.

💡 Example:  
A database transaction system might use checkpoints so that if a deadlock occurs, processes can rollback to a previously saved state without losing all progress.

Starvation - The Opposite of Deadlock  
Starvation happens when a process waits indefinitely because resources are always allocated to higher-priority processes.

💎 Solution to Starvation: Aging

Increase the priority of a waiting process over time so it eventually gets CPU access.  
Used in priority scheduling algorithms to prevent low-priority processes from being blocked forever.

💡 Example:  
A low-priority print job in a print queue might never print because high-priority documents keep arriving. Aging ensures the low-priority job eventually moves to the front of the queue.

Summary of Deadlock Handling Techniques

- ✓ Prevention - Modify the system to avoid at least one of the four conditions required for deadlock.
- ✓ Avoidance - Use Banker's Algorithm to dynamically ensure a safe state.
- ✓ Detection & Recovery - Detect deadlocks when they occur and recover by terminating processes, preempting resources, or rolling back processes.

💡 Real-World Applications:

Database Transactions: Use locking mechanisms to prevent deadlocks.  
Operating Systems: Use resource ordering and preemption to avoid deadlocks.  
Distributed Systems: Use timeouts to detect and break deadlocks.

-----  
-----  
-----

Operating Systems - Lecture 7: Memory Management & Main Memory

Why Memory Management is Important?

Memory is a finite resource, and the OS must manage it efficiently to ensure smooth process execution. Without proper memory management, processes may collide, overwrite data, or run out of memory.

💡 Example:

A computer with 8GB RAM running multiple applications (browser, game, editor, OS services) must allocate memory dynamically to each process without conflicts.

#### Key Memory Concepts

- ◆ The OS loads programs into memory from disk before execution.
- ◆ The CPU can directly access registers and main memory, but disk access is much slower.
- ◆ CPU registers are the fastest but have limited storage.
- ◆ Main memory (RAM) requires multiple CPU cycles for access, creating stalls (delays).
- ◆ Cache memory reduces the access delay by storing frequently used data closer to the CPU.
- ◆ Memory protection ensures processes do not access unauthorized memory regions.

#### 💡 Example:

A modern CPU (e.g., Intel i7) has multiple cache levels (L1, L2, L3) to bridge the speed gap between RAM and registers.

#### Key Terminology

- ◆ Segment - A portion of memory allocated to a process.
- ◆ Physical Address - The actual location in RAM.
- ◆ Logical (Virtual) Address - The address generated by the CPU for a process.
- ◆ Memory Management Unit (MMU) - A hardware unit that translates logical addresses into physical addresses.

#### 💡 Example:

A process requests data from address 0x200A → The MMU translates it to physical memory location 0x3F20A.

#### Memory Protection & Address Translation

- ◆ Base Register - Defines the starting address of a process in memory.
- ◆ Limit Register - Defines the maximum allocated memory range for a process.
- ◆ The CPU checks every memory access to ensure it stays within the base and limit registers.

#### 💡 Example:

A process with a base address of 1000 and a limit of 4000

Can access memory between 1000 and 5000.

If it tries to access 6000, the OS blocks the request.

Address Binding: When Are Addresses Assigned?

- ☐ Compile Time - If the memory location is known before execution, the compiler generates absolute addresses.

Example: An embedded system where programs always load at a fixed memory location.

📦 Load Time - The OS assigns addresses when loading the program into memory. Once assigned, the process cannot move.

📦 Execution Time (Dynamic Relocation) - The OS allocates and moves processes in memory dynamically.

💡 Example:

A game running on a PC needs dynamic relocation because the memory layout constantly changes as new processes start and exit.

### Memory Allocation Strategies

The OS allocates memory to processes using different strategies:

✓ First-Fit - Assigns the first available memory block that fits the process.

✓ Best-Fit - Assigns the smallest available memory block that fits the process.

✓ Worst-Fit - Assigns the largest available memory block, leaving maximum leftover space.

💡 Example:

If we have three free memory blocks of sizes 10MB, 20MB, and 30MB, and a process needs 12MB:

First-Fit → Allocates 20MB block.

Best-Fit → Allocates 20MB block (smallest possible fit).

Worst-Fit → Allocates 30MB block (largest block).

Memory Fragmentation: Wasted Space

💎 External Fragmentation - Occurs when enough total memory is available, but it is not contiguous, so a process cannot be allocated.

50% Rule: For every two allocated memory blocks, the third block is lost due to fragmentation.

💎 Internal Fragmentation - Occurs when a process is allocated more memory than it needs, leading to wasted space inside allocated blocks.

💡 Example:

A process needs 30MB, but is allocated 40MB → 10MB is wasted (internal fragmentation).

If 100MB is free, but divided into small, scattered blocks, a 50MB process cannot fit (external fragmentation).

Swapping - Freeing Memory for Active Processes

If RAM is full, the OS moves inactive processes to disk and brings them back when needed. This process is called swapping.

✓ Static Relocation: A swapped-out process must return to the same memory location.

✓ Dynamic Relocation: A swapped-out process can be loaded into a different memory location when reloaded.

### 💡 Example:

A paused video editing app is swapped out to free memory.  
When the user resumes the app, it is swapped back in.

Paging - Solving External Fragmentation

Paging divides memory into fixed-sized blocks (pages) to prevent external fragmentation.

- ✓ A process is divided into pages.
- ✓ Memory is divided into page frames of the same size.
- ✓ The OS maps pages to available frames, even if they are not contiguous.

◆ Rule 90/10: A process spends 90% of execution time using only 10% of its memory pages.

### 💡 Example:

If 4KB pages are used:

A 32KB process is divided into 8 pages.

These pages are placed in any available frames in RAM.

- ✓ Paging eliminates external fragmentation but requires a page table to keep track of mappings.

Page Tables & Translation Lookaside Buffer (TLB)

◆ Page Table - A mapping table that converts virtual addresses (CPU-generated) into physical addresses (RAM locations).

◆ TLB (Translation Lookaside Buffer) - A high-speed cache that stores frequently used page mappings, reducing translation delays.

### 💡 Example:

Without TLB: Every memory access requires two lookups (one for page table, one for data).

With TLB: The most frequently accessed pages are cached, speeding up memory access.

- ✓ Paging solves external fragmentation and allows process execution even with limited RAM.

Segmentation - Alternative to Paging

Instead of fixed-size pages, segmentation divides memory into logical sections based on program structure:

- ✓ Code Segment - Contains program instructions.
- ✓ Data Segment - Stores global variables.
- ✓ Stack Segment - Manages function calls and local variables.
- ✓ Heap Segment - Manages dynamically allocated memory.

◆ Difference Between Paging & Segmentation:

- ✓ Paging - Fixed-size memory blocks, efficient but may waste space.



✓ Segmentation - Variable-sized memory blocks, aligned with logical program structure.

💡 Example:

A video editor stores:

Editing functions in the code segment.

Video clips in the data segment.

Temporary effects in the stack segment.

✓ Segmentation eliminates internal fragmentation but requires compaction to avoid external fragmentation.

Hybrid Approach: Paging + Segmentation

◆ Combining paging & segmentation provides the best of both worlds:

✓ The OS allocates only necessary memory.

✓ Processes can use any free memory blocks.

✓ Two-level mapping allows better memory utilization.

💡 Example:

A game engine may use:

Segmentation to separate game logic, assets, and rendering pipelines.

Paging to allocate memory efficiently without fragmentation.

-----  
-----  
-----

Operating Systems - Lecture 8: Virtual Memory Management

What is Virtual Memory?

◆ Virtual Memory is a technique that allows a system to separate logical memory from physical memory, giving the illusion of a larger memory space than what is physically available.

◆ It enables programs to execute even when they do not fully fit in RAM.

💡 Example:

A computer with 8GB RAM can run a program that requires 16GB of memory by using virtual memory, which swaps data between RAM and disk storage dynamically.

Virtual Address Space

◆ The virtual address space represents how a process sees memory, which is different from the physical memory layout.

◆ The OS maps virtual addresses to physical memory locations dynamically.

✓ Implemented Using:

☐ Demand Paging - Loads only the required parts (pages) of a process into RAM when needed.

☐ Demand Segmentation - Loads segments instead of fixed-size pages.

💡 Example:

A web browser does not load the entire webpage into memory at once. Instead, it loads sections (pages) of the webpage on demand, improving efficiency.

Demand Paging - Efficient Use of RAM

◆ Instead of loading an entire process into RAM, the OS brings only necessary pages into memory when needed.

◆ Uses a lazy swapper to avoid loading unnecessary pages, reducing memory consumption.

✓ How It Works:

1️⃣ The process requests data from a page not in memory.

2️⃣ A page fault occurs, and the OS loads the required page from disk to RAM.

3️⃣ The program continues execution without noticing the delay.

💡 Example:

When you open a large PDF file, the OS loads only the visible pages, instead of the entire document.

Page Replacement Strategies - Choosing Which Page to Remove

When a page fault occurs and RAM is full, the OS must remove a page to free up space.

✓ Four Common Page Replacement Algorithms:

1️⃣ First-In-First-Out (FIFO)

Removes the oldest page first.

Simple but inefficient if frequently used pages are removed.

💡 Example:

A browser cache removes oldest tabs first when memory is full.

2️⃣ Least Recently Used (LRU)

Removes the page that has not been accessed for the longest time.

Better than FIFO, but requires tracking usage history.

💡 Example:

A smartphone OS removes least-used apps from RAM to free space.

3️⃣ Least Frequently Used (LFU)

Removes the least accessed page based on usage count.

💡 Example:

A streaming service caches popular movies and removes rarely watched content.

4️⃣ Random Replacement

Removes a random page.

Simple but unpredictable.

💡 Example:

Used in some real-time systems where precise tracking is not feasible.

Why is Virtual Memory Important?

- ✓ Enables large applications to run on small RAM.
- ✓ Improves multitasking by efficiently managing memory allocation.
- ✓ Prevents fragmentation by using paging or segmentation techniques.

💡 Real-World Use Case:

Gaming consoles use virtual memory to load large game environments dynamically.

Cloud servers allocate memory to virtual machines dynamically based on demand.

-----  
-----  
-----

Operating Systems - Lecture 9: Storage System Structure

How Does a Disk Work?

A hard disk consists of multiple platters, each with read/write heads that move across the disk's surface to store and retrieve data.

- ✓ Data is stored on circular tracks that are further divided into sectors.
- ✓ A set of tracks aligned vertically across platters forms a cylinder.
- ✓ Disk access time depends on:

Seek time - Time for the head to move to the correct track.

Rotational latency - Time for the sector to rotate under the read/write head.

Data transfer time - Time taken to transfer data to/from disk.

💡 Example:

When you open a file, the OS calculates where it is stored, moves the disk head to that location, waits for the correct sector to rotate under the head, and then reads the data into RAM.

Disk Scheduling Algorithms

Since multiple processes compete for disk access, the OS schedules disk requests to minimize seek time.

- ✓ First-Come, First-Served (FCFS)

Handles requests in the order they arrive.

Fair but inefficient, as it may cause long seek times.

💡 Example:

Requests arrive in the following order: 98, 183, 37, 122, 14, 124, 65, 67

FCFS executes them in this order, leading to high head movement.

### ✓ Shortest Seek Time First (SSTF)

Picks the closest request to the current head position.  
Faster than FCFS but can cause starvation if far requests wait too long.

💡 Example:

If the disk head is at 50 and requests are 10, 20, 60, 80, SSTF picks 60 first, then 80, 20, and 10.

### ✓ SCAN (Elevator Algorithm)

The disk head moves in one direction, serving all requests, then reverses.  
Prevents starvation but may favor middle requests.

💡 Example:

If requests are 10, 50, 180, 190 and the head starts at 100, SCAN serves:  
50 → 10 → 100 → 180 → 190.

### ✓ C-SCAN (Circular SCAN)

Moves one way, resets to the beginning, and repeats.  
Ensures fair access to all requests.

💡 Example:

If head is at 100, it moves towards 200, resets to 0, and continues scanning.

## Storage Optimization Techniques

### ◆ Interleaving (Non-Sequential Allocation)

Blocks are not stored sequentially but are spaced apart to match disk access speeds.

Prevents bottlenecks when reading large files.

### ◆ Read Ahead (Prefetching)

The disk controller loads upcoming blocks into cache before the request arrives.

Speeds up sequential file access.

💡 Example:

A video player preloads the next few seconds of a video to prevent buffering.

## Error Detection & Correction

✓ Error Detection: Identifies corrupt data on storage devices.

✓ Error Correction: Fixes recoverable errors before they impact the system.

### ◆ Parity Bit

Adds an extra bit to detect errors.

If a single bit flips, the system detects the error but cannot fix it.

### ◆ Cyclic Redundancy Check (CRC)

Uses hash functions to check if data is corrupted.

More accurate than parity bits.

#### ◆ Error Correction Code (ECC)

Can detect and correct soft errors (temporary data corruption).

Hard errors (physical damage) can only be detected, not fixed.

💡 Example:

ECC memory in servers automatically fixes small memory errors to prevent crashes.

#### Storage Formatting & File System Management

##### ✓ Low-Level Formatting (Physical Formatting)

Divides disk into sectors before data storage is possible.

##### ✓ Root Partition

Stores the operating system and essential files.

##### ✓ Swap Space

A dedicated disk area used for paging and virtual memory.

💡 Example:

If RAM is full, inactive data is moved to swap space to free memory for active processes.

#### Network Storage & Redundancy

##### ◆ Network-Attached Storage (NAS)

Allows multiple devices to access shared storage over a network.

Uses protocols like NFS and CIFS.

##### ◆ Redundant Array of Independent Disks (RAID)

Combines multiple disks to improve performance and fault tolerance.

##### ✓ RAID Benefits:

RAID 0 - Striping for speed, no redundancy.

RAID 1 - Mirroring for data protection.

RAID 5 - Striping with parity for fault tolerance.

##### ◆ Snapshot

Captures the state of a file system before making changes.

Used in backup systems.

💡 Example:

Before updating an OS, a snapshot is taken so the system can roll back if something fails.

-----  
-----  
-----

◆ The hardware provides:

Durability - Data remains stored even after power loss.

Speed - Fast read/write operations.

◆ The Operating System (OS) ensures:

Storage capacity - Allocating disk space efficiently.

Access control & security - Restricting unauthorized access.

Ease of use - Providing a structured interface for file operations.

💡 Example:

A database system requires fast access, reliability, and security, while a log file system needs continuous writing capabilities.

What is a File?

◆ A file is a logical unit of storage on a disk, containing data or programs.

✓ File Attributes:

Name - Unique identifier.

Type - Text, binary, executable, etc.

Location - Disk address of the file.

Size - File length in bytes.

Permissions - Who can read/write/execute.

Timestamps - Creation, modification, and last accessed time.

💡 Example:

A Word document has attributes like name (report.docx), type (text), size (2MB), and last modified time.

Common File Operations (System Calls)

✓ Data Operations:

create() - Create a new file.

open() - Open an existing file.

read() - Read from a file.

write() - Write to a file.

delete() - Remove a file.

close() - Close an opened file.

seek() - Move to a specific position in a file.

✓ Metadata Operations:

Retrieve and update file attributes (size, timestamps, permissions).

💡 Example:

Media players use seek() to jump to a specific part of a video.

Operating systems maintain metadata such as file access history for security purposes.

Open File Tables - How the OS Manages Open Files

✓ Global Open File Table (Shared by All Processes):

Stores attributes of all opened files.

Tracks the file's disk location.

Contains file pointers to track read/write position.

✓ Per-Process Open File Table:

Each process has its own entry pointing to the global file table.

Tracks the process-specific read/write position.

Defines access mode (read, write, execute).

💡 Example:

When two programs open the same file, the global table ensures consistency, while each process maintains its own file pointer.

File Access Methods

◆ Sequential Access

Data is processed in order, byte by byte.

Best for text files, logs, and media streaming.

💡 Example:

Reading a book from page 1 to the last page.

◆ Direct Access (Random Access)

Data is accessed at any position, using an offset.

Used for databases and indexed files.

💡 Example:

Jumping to a specific scene in a movie file without playing from the start.

Directory Structures

✓ Single-Level Directory

All files stored in a single location.

Simple but lacks organization.

💡 Example:

MS-DOS (old systems) used a flat file system with no subdirectories.

✓ Two-Level Directory

Each user has their own directory.

Prevents file name conflicts between users.

💡 Example:

Windows User Profiles (C:\Users\Alice\Documents and C:\Users\Bob\Documents).

✓ Multi-Level Directory (Hierarchical Structure)

Uses a tree-like structure with subdirectories.

Provides better organization and access control.

💡 Example:

UNIX/Linux file system follows /home/user/Documents/notes.txt.

✓ Hard Links vs. Soft Links

Hard Link - Points directly to the same file inode (acts as a second name).

Soft Link (Symbolic Link) - Points to a file name (can exist across directories).

💡 Example:

Hard links are used for backups where files need multiple references.

Soft links are used for shortcut files on Windows (.lnk) and Linux (ln -s).

Disk Performance & Efficiency Metrics

✓ Disk Overhead (Processing Time)

Time the CPU spends processing a disk request before execution.

✓ Disk Latency (Access Time)

Seek Time - Time taken for the disk head to reach the correct track.

Rotation Time - Time taken for the sector to rotate under the head.

✓ Transfer Rate

The speed at which data moves from the disk to memory.

💡 Example:

SSD vs. HDD - SSDs have lower seek time and rotational delay because they have no moving parts.

File System Implementation & Memory Similarities

✓ Continuous Allocation

Simple, but causes external fragmentation.

Requires defragmentation to optimize performance.

✓ Indexed Allocation

Uses an index table (similar to paging) to map file blocks to disk locations.

✓ Free Space Management

Bitmap - Uses a bit array to track free/occupied blocks.

Linked List - Stores free blocks as a linked list.

💡 Example:

Windows NTFS uses bitmaps, while Linux ext4 uses a mix of bitmaps and lists.

-----  
-----  
-----

Operating Systems - Lecture 11: Virtualization & Cloud Computing

What is Virtualization?

💠 Virtualization is the process of creating a virtual (software-based) version of a physical resource, such as an operating system, storage, or network. It allows multiple independent environments to run on a single physical system.



## ✓ Types of Virtualization:

Type 1 / Bare Metal Hypervisor - Runs directly on hardware and manages virtual machines without needing an OS.

Type 2 / Hosted Hypervisor - Runs on top of an existing OS (e.g., VMware Workstation, VirtualBox).

💡 Example:

A cloud provider (AWS, Google Cloud) uses Type 1 Hypervisors to host thousands of virtual machines (VMs).

Developers use Type 2 Hypervisors on their laptops to run test environments.

What is a Hypervisor?

◆ A Hypervisor (Virtual Machine Manager - VMM) is a software layer that allows multiple operating systems (OS) to share the same hardware efficiently.

## ✓ Hypervisor Responsibilities:

Allocates CPU, memory, and storage to virtual machines.

Ensures isolation between VMs to prevent conflicts.

Supports live migration (moving a VM between physical machines).

💡 Example:

KVM, VMware ESXi, Microsoft Hyper-V, and Xen are examples of hypervisors used in enterprise cloud computing.

What is Cloud Computing?

◆ Cloud Computing is an on-demand service model that provides scalable computing resources (servers, storage, and applications) over the internet.

## ✓ Key Attributes of Cloud Computing:

Scalability - Dynamically increases/decreases resources based on demand.

Low Latency - Fast response times for user requests.

Guaranteed Performance - Provides consistent resource availability.

Extensible Management - Can be monitored and controlled remotely.

Self-Healing Resilience - Automatically detects and recovers from failures.

💡 Example:

Netflix uses cloud services to scale video streaming globally while ensuring smooth performance.

Essential Characteristics of Cloud Computing

## ✓ On-Demand Self-Service

Users can provision computing resources automatically without human intervention.

💡 Example: AWS EC2 instances can be launched and terminated at any time.

## ✓ Broad Network Access

Services are accessible from any device over the internet.

💡 Example: Google Drive allows users to access files from desktops, tablets, and smartphones.

✓ Resource Pooling

Cloud providers dynamically allocate resources to multiple customers.

💡 Example: AWS dynamically assigns server resources based on customer workloads.

✓ Rapid Elasticity

Resources can be scaled up or down automatically.

💡 Example: A retail website increases cloud server capacity during Black Friday sales.

✓ Measured Service

Customers pay based on actual resource usage.

💡 Example: AWS charges for storage (S3) and computing power (EC2) based on usage time.

Cloud Service Models

◆ Infrastructure as a Service (IaaS)

Provides virtualized computing resources over the internet.

Users can rent virtual machines, storage, and network resources.

💡 Example: AWS EC2, Google Compute Engine, Microsoft Azure VMs.

◆ Platform as a Service (PaaS)

Provides a ready-to-use software development environment.

Developers get databases, application servers, and middleware without managing infrastructure.

💡 Example: Google App Engine, AWS Lambda, Microsoft Azure App Services.

◆ Software as a Service (SaaS)

Delivers fully functional applications via the internet.

Users don't need to install or maintain software.

💡 Example: Google Workspace (Docs, Gmail), Microsoft 365, Dropbox.

-----  
-----  
-----

Operating Systems - Additional Exam Points & Key Concepts

Execution of Programs

✓ Operating System Service → The OS provides execution environments for programs.

✓ Parallelism vs. Concurrency

A system is parallel if it can execute multiple processes at the same time.

Concurrency means multiple tasks progress simultaneously, but not necessarily at the same time.

💡 Example:

Multithreaded applications execute concurrent tasks (e.g., a web browser loading multiple pages).

Multi-core CPUs execute truly parallel processes.

Process Management Activities

✓ The OS handles process management by:

Suspending and resuming processes.

Creating and deleting user/system processes.

Providing process synchronization mechanisms.

Creating/terminating processes.

Facilitating interprocess communication.

Handling deadlocks.

💡 Example:

OS Process Table keeps track of all running and suspended processes.

Types of System Calls

✓ System Calls are categorized as:

Process Control - Create, terminate, wait for a process.

File Management - Open, close, read, write, delete files.

Device Management - Request/release a device.

Information Maintenance - Time, date, process status.

Communications - Interprocess communication via messages.

💡 Example:

Linux system call: `fork()` creates a child process.

Scheduling Strategies & CPU Execution

✓ Preemptive Scheduling → Temporarily suspends processes and resumes them later.

✓ CPU Scheduling Algorithms

First-Come, First-Served (FCFS)

Shortest Job First (SJF)

Priority Scheduling

Round Robin (RR) - Each process gets a fixed time slice

💡 Example:

Round Robin (RR) gives each process 10-100ms before switching.

Memory & Address Translation

✓ The OS maintains a separate PAGE table for each process.

✓ Memory Allocation Strategies:

First-Fit - Allocates the first free block large enough for the process.

Best-Fit - Allocates the smallest possible block.

Worst-Fit - Allocates the largest available block.

💡 Example:

Paging eliminates external fragmentation and allows processes to execute with partial memory availability.

Operating System Operations & Resource Allocation

### ✓ OS Functions

Process Management - Controls process creation, execution.

I/O Device Control - Manages keyboard, mouse, storage.

Resource Allocation - Ensures optimal CPU/memory usage.

### 💡 Example:

Windows Task Manager monitors CPU and RAM utilization.

Program vs. Process

✓ A program is a passive entity, a process is an active execution of a program.

### ✓ Process States:

New - Being created.

Ready - Waiting for CPU.

Running - Currently executing.

Waiting - Suspended for an event.

Terminated - Finished execution.

### 💡 Example:

A running browser process loads multiple threads for tabs, extensions, and background tasks.

Thread vs. Process

✓ Creating a thread consumes fewer resources than creating a process.

### 💡 Example:

Threads share memory within a process, while processes require separate memory space.

File System Management

✓ Files have attributes such as:

Name, type, location, size, and protection levels.

✓ Two file access methods:

Sequential Access - Data is processed in order.

Direct Access - Specific bytes or blocks can be accessed directly.

### 💡 Example:

Database systems use direct access, while log files use sequential access.

Disk Scheduling Algorithms

✓ Disk I/O Scheduling methods:

First-Come, First-Served (FCFS)

Shortest Seek Time First (SSTF)

SCAN & C-SCAN

💡 Example:

FCFS is fair but slow, SSTF reduces seek time but may cause starvation.  
Interprocess Communication (IPC)

✓ Two models of IPC:

Shared Memory - Processes share a memory segment.

Message Passing - Processes communicate via messages.

✓ Blocking vs. Non-blocking Communication:

Blocking - Sender/receiver waits for a response.

Non-blocking - No waiting required.

💡 Example:

Sockets enable IPC across networked machines.

Deadlocks & Resource Allocation

✓ Four necessary conditions for deadlock:

Mutual Exclusion - A resource is held by only one process at a time.

Hold and Wait - A process holding resources is waiting for additional ones.

No Preemption - Resources cannot be forcibly taken from a process.

Circular Wait - A circular chain of processes waiting on resources.

💡 Example:

Banker's Algorithm prevents deadlock by checking safe states.

Virtualization & Cloud Computing

✓ Virtualization Types:

Type 1 Hypervisor (Bare Metal) - Directly on hardware (e.g., VMware ESXi).

Type 2 Hypervisor (Hosted) - Runs within another OS (e.g., VirtualBox).

✓ Cloud Computing Models:

IaaS (Infrastructure as a Service) - Provides virtualized hardware.

PaaS (Platform as a Service) - Offers development platforms.

SaaS (Software as a Service) - Delivers applications over the internet.

💡 Example:

AWS EC2 is IaaS, Google App Engine is PaaS, and Gmail is SaaS.

RAID (Redundant Array of Independent Disks)

✓ RAID 0 requires at least 2 disks.

💡 Example:

RAID 1 mirrors data for redundancy, RAID 5 stripes with parity for fault tolerance.